

Persistent Agent Context with Progressive Retrieval Attention: Typed Resource Discovery Before Neural Materialization

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 25, 2026

Abstract

Agent runtimes often place every tool definition in every prompt. This makes a large but mostly inactive catalog consume native context before the model has decided which resource it needs. We separate that decision into a typed resource-discovery layer for Progressive Retrieval Attention (PRA). The layer supports explicit, lexical, indexed, semantic, hybrid, hinted, and adaptive policies; preserves discovery provenance; and turns calibrated confidence into select, ask, or abstain decisions. We evaluate the deterministic M0 mechanism on opaque synthetic catalogs from 8 to 8,192 resources, five seeds, six positive reference forms, and ambiguous and nonexistent controls. At 8,192 resources, both request hints and adaptive fallback obtain 100% top-1 selection, 100% actionable outcome accuracy, and no false action on the ten negative cases. Hints require 1.08 retrieval stages on average, compared with 2.39 for adaptive fallback. A fixed hybrid policy also ranks every target first, but acts on 98.3% of positive requests and has the largest policy-cost regret. An unguided automatic policy ranks five of six positive forms correctly but acts on none after calibration, showing that ranking and safe use are different objectives. An 8,192-resource catalog contains 147,456 logical definition tokens while a selected definition contains 18 tokens (0.0122%); this is accounting evidence, not yet native-K/V or end-to-end agent evidence. The results establish the discovery mechanism and identify production indexing, neural materialization, and tool execution as separate gates.

1 The Agent Catalog Problem

An append-only agent prompt couples the size of the logical environment to the transformer’s active context. Tool schemas, skills, system instructions, artifacts, and old observations are repeatedly exposed even when a turn uses only one of them. Compaction can reduce prompt length, but it may erase provenance or stale-versus-current distinctions.

PRA suggests a different decomposition:

agent state = bounded native workspace + persistent addressable resources.

This decomposition does not make storage free. Source text, indexes, cached representations, resident K/V, transferred K/V, and attention-visible K/V are different costs. It instead introduces a concrete question before materialization: which resource identity should be resolved, by which discovery mechanism, and with enough confidence to act?

This paper reports the first mechanism gate. It asks whether a typed resolver can preserve selection quality as a catalog grows, whether policy hints reduce unnecessary search, and whether confidence prevents an accurate ranker from becoming an unsafe selector. It does not yet claim improved model generation, tool execution, persistent-session recall, or native-K/V savings.

2 Typed, Versioned Resources

A resource has a stable typed identity, versioned source, aliases, schema, side-effect class, tenant, and revocation state. Representative identifiers are

```
!!ref:tool:calendar:create_event:v3!!  
!!ref:skill:paper-review:v2!!  
!!ref:artifact:experiment-17!!  
!!ref:session:parent/decision-14!!
```

Identity is distinct from text, index entries, semantic summaries, cached K/V, and current residency. A cache fingerprint therefore includes source/version, model, tokenizer, routing configuration, and any position-sensitive encoding state.

The implementation in `src/prahf/agent_resources.py` enforces tenant, revocation, and allowed-URI filters before ranking. Discovery returns a trace, not only a winner: the executed channel path, per-candidate provenance, confidence, fallback count, and final select/ask/abstain decision remain available to the caller. The resolver never authorizes or executes a tool.

3 Discovery Channels and Policy

The SDK exposes

$$\pi \in \{\text{auto}, \text{explicit}, \text{token}, \text{index}, \text{semantic}, \text{hybrid}, \text{adaptive}\}.$$

Hints can be attached to a request or inherited from a resource collection. Strict hints execute one channel. Non-strict hints specify a starting point and permit calibrated fallback.

Explicit. A typed URI is resolved by exact map lookup. This is deterministic after access checks.

Token. Names, aliases, descriptions, and token n-grams are compared lexically. The M0 implementation is deliberately exhaustive and therefore a correctness control, not a production latency path.

Index. Persistent exact, token-posting, n-gram, and BM25 structures narrow the candidate set before scoring. Index fingerprints make freshness auditable.

Semantic. M0 uses a declared 96-dimensional signed-hash control over the resource’s action, object, and family concepts. It contains no resource ID. This idealized control isolates the policy mechanism; it is not evidence about a learned embedding model.

Hybrid and adaptive. Hybrid fuses all available evidence in one expensive stage. Adaptive begins cheaply and escalates only when calibrated confidence is insufficient. A representative ladder is

$$\text{explicit} \rightarrow \text{token} \rightarrow \text{index} \rightarrow \text{semantic} \rightarrow \text{hybrid} \rightarrow \text{ask/abstain}.$$

The concrete path is recorded per request.

3.1 Confidence is part of correctness

For candidate r_i , the calibrator estimates

$$c_i \approx P(r_i \text{ is intended} \mid q, \mathbf{s}_i, p_i),$$

where $\mathbf{s}_i$ denotes channel scores and p_i their provenance. High confidence permits selection; intermediate confidence asks or searches more; low confidence abstains. For a positive request, the actionable outcome is correct only when the right resource is ranked first *and* the decision is **select**. For an ambiguous or nonexistent request, it is correct only when the decision is **ask** or **abstain**. This definition prevents a low-confidence ranker from receiving operational credit merely because its latent top candidate was right.

4 M0 Experimental Design

4.1 Opaque catalogs

The generator in `src/data/agent_resources.py` creates identifiers such as `tool_zaf_193`. Each resource combines held-out action, object, and family concepts with aliases, a schema, a version, and a side-effect class. The opaque names prevent pretrained knowledge from solving the benchmark. Catalogs contain 8, 32, 128, 512, 2,048, and 8,192 resources.

Each held-out identity produces six positive requests: explicit URI, exact name, alias, typo, semantic paraphrase, and descriptive reference. Each split also contains one ambiguous and one nonexistent request. Validation identities calibrate thresholds; test identities are disjoint. At sizes 32–8,192, each policy is evaluated on 540 positive and 10 negative test requests across seeds {11, 23, 37, 53, 71}. The 8-resource condition uses 180 positive and 10 negative requests. The complete artifact contains 31,680 validation and test policy traces.

4.2 Policies and metrics

We compare five fixed policies (explicit, token, index, semantic, and hybrid), unguided **auto**, a request-level hint, and adaptive fallback. The oracle is the least-cost fixed policy that ranks the target first on that query. Reported metrics include top-1 accuracy, selection coverage, selective and actionable outcome accuracy, safe nonaction and false-action rates, retrieval stages, fallback count, Brier score, expected calibration error (ECE), quality regret, and policy-cost regret. Confidence thresholds are fitted only on the validation split. Intervals are percentile bootstrap intervals over five seed means; deterministic outcomes can consequently have zero-width intervals.

4.3 Systems accounting

Cold index construction and a full immutable rebuild after mutating one percent of resource versions are timed separately. Warm component latency is the mean of repeated CPU Python calls. Policy-path latency is reconstructed by summing the measured components actually executed by each trace. These timings exclude tokenization, a model forward pass, native-K/V encoding or transfer, attention, and tool execution. They characterize this reference implementation, not an optimized serving system.

5 Results

5.1 Selection scales, but policies differ operationally

Table 1 reports the largest catalog. Hinted and adaptive policies select every positive target and safely reject every negative request. Their equal quality hides a meaningful systems difference: hints reduce the mean path from 2.39 stages to 1.08 and the measured component estimate from 1,084 to 691 ms per request. The fixed hybrid ranker also attains 100% top-1, but calibration leaves 1.7% of positive requests unselected. Its exhaustive 1,405 ms component cost and 4.17 cost-regret units make it a poor default.

Table 1: Held-out M0 results at 8,192 resources. Top-1 is computed on 540 positive requests; outcome includes ten ambiguous/nonexistent controls. Time is reconstructed CPU Python discovery-component time.

Policy	Top-1	Coverage	Outcome	Safe neg.	Stages	ms/query
Explicit	.167	.167	.182	1.000	1.00	5
Token	.713	.537	.545	1.000	1.00	1,174
Index	.833	.000	.018	1.000	1.00	27
Semantic	.333	.000	.018	1.000	1.00	339
Hybrid	1.000	.983	.984	1.000	1.00	1,405
Auto	.833	.000	.018	1.000	1.00	24
User hint	1.000	1.000	1.000	1.000	1.08	691
Adaptive	1.000	1.000	1.000	1.000	2.39	1,084

The unguided automatic policy exposes why ranking is insufficient. It ranks explicit, name, alias, typo, and description requests correctly, misses the semantic-paraphrase stratum, and obtains .833 top-1. Calibration nevertheless causes it to ask on every positive request, so its actionable outcome accuracy is only $2/110 = .018$ per seed: the two safe negative decisions. Fixed index has the same operational pattern. Conversely, fixed token acts selectively and is always right when it acts, but covers only .537 of positive requests.

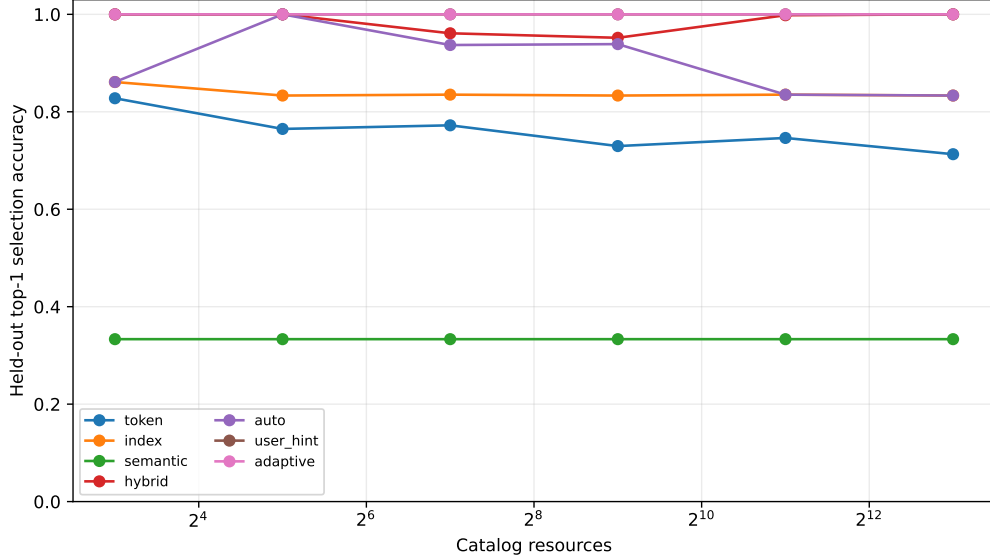


Figure 1: Positive-request top-1 accuracy as catalog size increases. Ranking alone does not encode whether confidence permits selection.

Figure 1 shows that hints and adaptive fallback maintain 1.0 top-1 at every tested size. The result is not uniformly safe at the smallest size: in the 8-resource condition each policy attains 1.0 positive top-1, but hinted and adaptive routing each falsely select the ambiguous held-out case in all five seeds. Their overall outcome accuracy is .974 and safe-nonaction rate is .5. From 32 resources onward, both negative cases are handled correctly in all seeds. This inversion is a calibration artifact of the controlled score distribution and cautions against treating larger catalogs as automatically harder along every axis.

5.2 No fixed channel is the oracle everywhere

At 8,192 resources, the least-cost correct oracle chooses explicit lookup for 16.7% of positive requests, indexed lookup for 66.7%, and semantic lookup for 16.7%. Neither token nor hybrid is ever the least-cost correct choice. User hints have zero quality regret and 0.083 cost-regret units, compared with zero quality regret and 0.5 cost regret for adaptive fallback. This supports a narrow claim: when callers know the reference form, passing that information avoids discovery work; adaptive escalation remains useful when they do not.

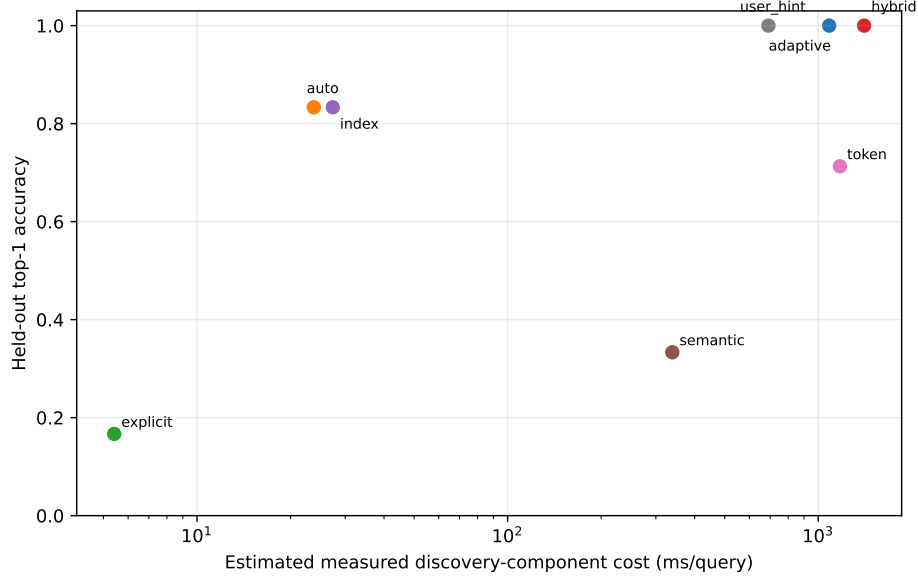


Figure 2: Largest-catalog ranking quality versus measured discovery-component cost. The log axis is needed because explicit and exhaustive hybrid paths differ by more than two orders of magnitude.

5.3 Logical catalog size is not active materialization

Every generated definition contains 18 accounting tokens. The largest catalog therefore contains 147,456 logical definition tokens, whereas resolving one resource requests 18 tokens, or 0.0122% of the catalog. Figure 3 makes that distinction visible. The number is an accounting upper bound on selected source text; M0 does not encode definitions into native K/V and therefore does not establish attention-memory or latency savings.

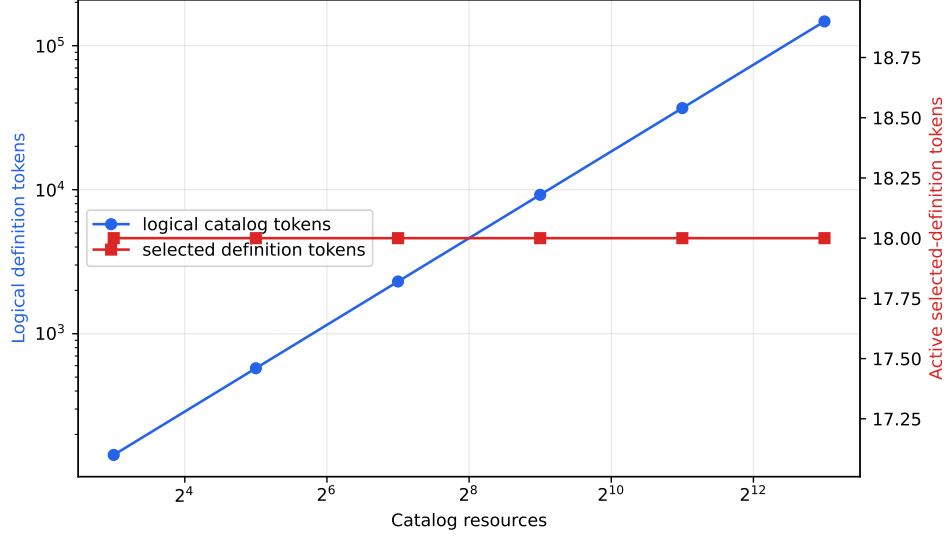


Figure 3: Logical catalog definitions grow with catalog size while the one-item selected-definition accounting remains at 18 tokens. The axes intentionally separate logical storage from active materialization.

5.4 Persistent indexing helps, but is not yet production-ready

At 8,192 resources, index construction takes 1.727 s, a full rebuild after a one-percent version mutation takes 1.753 s, and the estimated Python index payload is 5.27 MiB. Warm indexed lookup takes 27.4 ms and scores 966 candidates on average (11.8% of the catalog). By comparison, exhaustive semantic, token, and hybrid controls take 339, 1,174, and 1,405 ms. The index build amortizes to 44.6 ms/query after 100 uses and 29.1 ms after 1,000 uses.

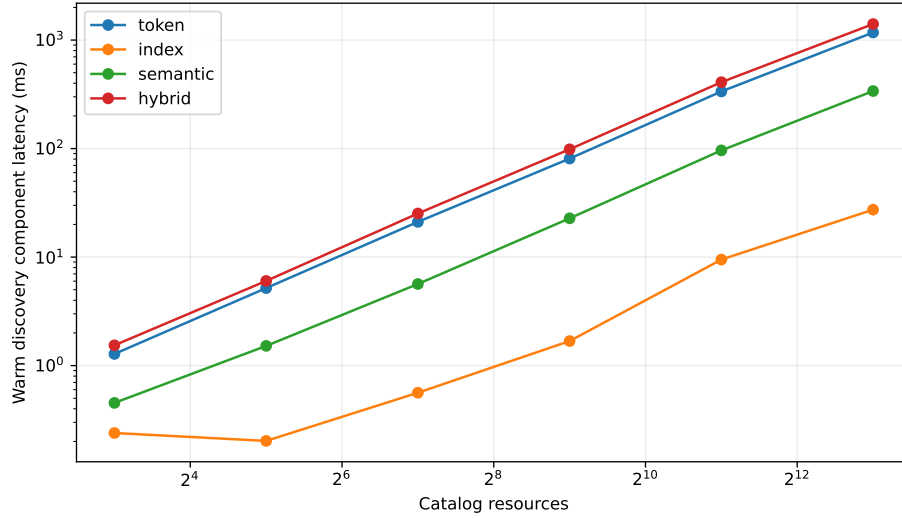


Figure 4: Warm CPU Python component latency. Exhaustive channels scale poorly; the indexed path narrows candidates but still touches about 11.8% at 8K.

These numbers establish an optimization target rather than a speed claim. Production token postings should avoid Python object traversal, semantic search should use an approximate vector index, unions should be deduplicated before scoring, and mutation should update affected postings rather than rebuild the entire immutable index. Cold/warm serving measurements must also include tokenization and downstream native-K/V work.

6 What the M0 Gate Establishes

The mechanism satisfies four narrow requirements. First, typed resources can be resolved without exposing the full catalog to a language model. Second, different reference forms favor different channels. Third, request hints can match adaptive quality with fewer stages. Fourth, action-aware calibration can distinguish a correct ranking from a decision safe enough to use.

The experiment does *not* yet satisfy the paper’s broader context gate. The 18-token active count is not a measured model working set. There is no native-K/V materialization, argument generation, execution, observation, persistent-history recall, skill composition, or subagent inheritance. The negative controls are small, and the semantic control is idealized. Accordingly we describe M0 as resolver evidence, not evidence that PRA improves agents.

7 Next Experimental Gates

M1: causal toy model. Train a small decoder on opaque tool-use language so semantics exist only in the supplied definitions. Compare eager definitions with discover–materialize–call under matched active-definition budgets.

M2: pretrained bridge. Use Qwen3-0.6B for schema/argument generation after a fixed resource identity has been selected. Run disabled, shuffled, irrelevant, empty, and oracle materialization controls. No side-effect executes unless host authorization and an execution-specific gate both pass.

M3: replication and realistic catalogs. Replicate selected conditions on another tokenizer/model family. Replace the idealized semantic control with declared embedding baselines and add naturally ambiguous resource names.

Persistent skills and sessions. After tool selection passes, test hierarchical skill disclosure, version mutation and revocation, bounded `#_head` history, superseded facts, and permission-scoped child sessions

$$M_{\text{child}} = R(M_{\text{parent}}) \cup \Delta M_{\text{child}}.$$

The OpenAI-compatible endpoint belongs after these mechanism checks, so harness behavior does not hide resolver or materialization failures.

8 Reproducibility

The benchmark entry points are `experiments/paper6.5_tools/run_m0_policy_study.py` and `summarize_m0_policy_study.py`. Machine-readable outputs include the manifest, one row per policy/query trace, index costs, seed summaries, stratum summaries, oracle distributions, and plots under `docs/papers/shared/results/paper6.5_tools`. The recorded environment is

Python 3.10.11 on Windows, Intel64 Family 6 Model 94. All reported M0 results are deterministic conditional on the five catalog seeds.

9 Conclusion

A large agent catalog should not be mistaken for the context needed on every turn. In this controlled M0 study, typed discovery resolves one resource from 8,192 while keeping the selected-definition accounting at 18 tokens. Hinted and adaptive policies attain perfect largest-catalog selection and safe nonaction, but hints use less search. The same study also produces two useful negative findings: an accurate unguided ranker can be too uncertain to act, and small-catalog ambiguity can defeat calibration even when positive selection is perfect. These results justify proceeding to neural materialization and tool use, but they do not substitute for those experiments.